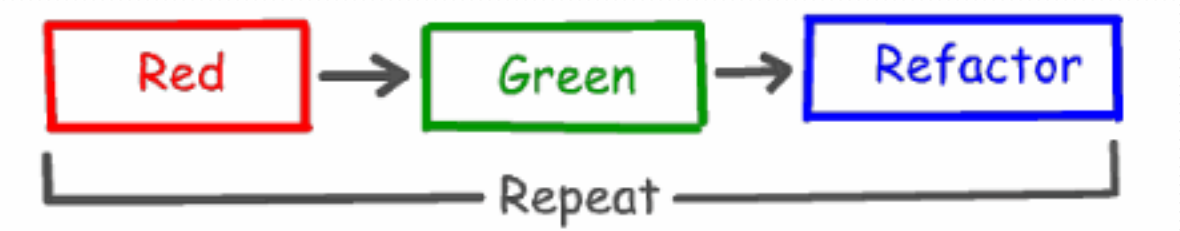
Разработка ПО через тестирование

Модульное тестирование и Test-Driven Development

- тестирование - это хорошо, значит, программисты должны постоянно тестировать свой код.
- TDD - это методика разработки, позволяющая оптимизировать использование модульных тестов
- Задача, которую преследует TDD, - достижение баланса между усилиями и результатом.

цикл "red/green/refactor"



- В первой фазе программист пишет тест,
- во второй - код, необходимый для того, чтобы тест работал,
- в третьей, при необходимости, производится рефакторинг.
- В соответствии с принципом "Test First", следует писать только такой код, который абсолютно необходим, чтобы тесты выполнялись успешно.

Необходимо написать метод,
преобразующий числа от 1 до 7 в названия соответствующих дней недели.

- пишем тест для этого метода

```
[Test, ExpectedException(typeof(ArgumentException))]
public void TestGetDayOfWeekNameInvalidArgument()
{
  Converter.GetDayOfWeekName(8);
}
```

- создаем метод-заглушку на тестируемом классе, необходимую для того, чтобы проект собрался

```
static public string GetDayOfWeekName(int dayNumber)
{
 return string.Empty;
}
```

Необходимо написать метод, преобразующий числа от 1 до 7 в названия соответствующих дней недели.

- Заглушку следует писать так, чтобы тест не выполнялся - это поможет удостовериться, что тест правильно реагирует на ошибку.
- задача ***"красной" фазы*** состоит в том, чтобы в процессе написания теста определить сигнатуры тестируемых методов, порядок их вызова и другие особенности работы с ними.
- имеет смысл начинать тестирование метода с наиболее общей, дефолтной ситуации (в метод подаются числа, выходящие за пределы заданного диапазона).
- Поэтому мы пишем тест, который закончится удачно только в том случае, если тестируемый метод выбросит исключение заданного типа.

Необходимо написать метод, преобразующий числа от 1 до 7 в названия соответствующих дней недели.

- В ***"зеленой" фазе*** мы имплементируем и отлаживаем наш метод. При наличии альтернатив имеет смысл использовать самую простую имплементацию. Если возникает необходимость написать код, который не проверяется тестом, необходимо вернуться в красную фазу и исправить тесты.

```
static public string GetDayOfWeekName(int dayNumber)
 {
        throw new ArgumentException();
 }
```

- Добившись успешного выполнения всех написанных тестов, просматриваем код в поисках потенциально полезных рефакторингов.

- завершения цикла "red - green - refactor" его нужно повторить для следующего участка функциональности.

Необходимо написать метод,
преобразующий числа от 1 до 7 в названия соответствующих дней недели.

“red”

```
[Test] public void TestGetDayOfWeekName()
{
        Assert.AreEqual("Monday", Converter.GetDayOfWeekName(1));
        …
        Assert.AreEqual("Saturday", Converter.GetDayOfWeekName(7));
}
```

“green”

```
static public string GetDayOfWeekName(int dayNumber)
{
        switch (dayNumber)
        {
                case 1: return "Monday";
                …
                case 7: return "Saturday";
        }
        throw new ArgumentException();
 }
```

Необходимо написать метод,
преобразующий числа от 1 до 7 в названия соответствующих дней недели.

“refactor”

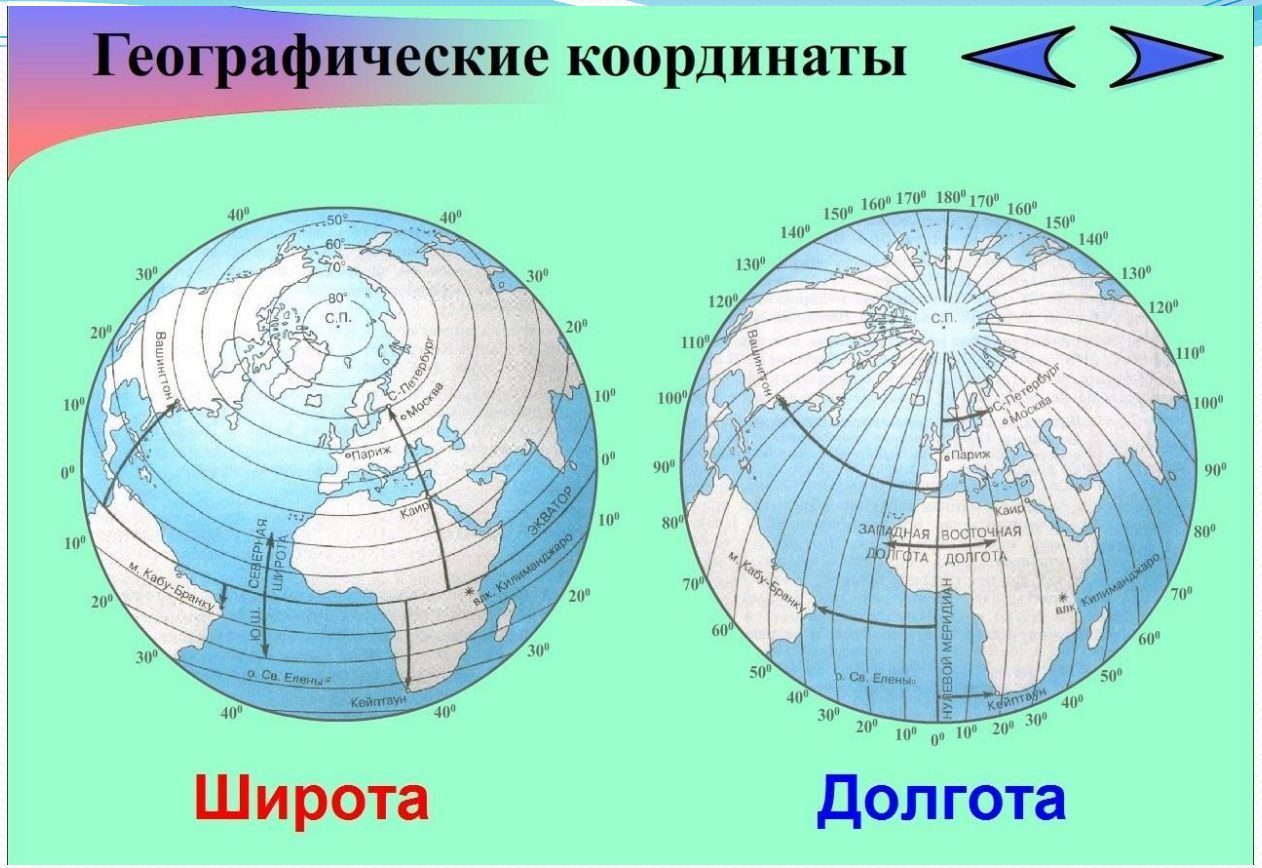
```
static public string GetDayOfWeekName(int dayNumber)
{
        Hashtable ht = new Hashtable();
        ht[1] = "Monday";
        ht[7] = "Saturday";
        string result = ht[dayNumber] as string;
        if (result == null)
                throw new ArgumentException();
        return result;
}
```

- "red - green - refactor" должен занимать от 5 до 20 минут
- Не следует работать в нескольких фазах одновременно: рефакторить код, например, в ***"красной" фазе*** неразумно.
- Чем больше тестов отлаживается в одном цикле, тем хуже; в идеале нужно работать только с одним тестом одновременно.

Задание.

- Используя методологию **TDD** разработать класс, который вычисляет расстояние и азимут между двумя точками на Земном шаре. Последовательность цикла "red/green/refactor" не удаляем, а помещаем в комментарий. Это понадобится для отчета.
- Азимут – это угол, между направлением на географический северный полюс (не магнитный) и направлением из точки 1 в точку 2

Географические координаты



- **Входными** данными являются долгота и широта 1 и 2 точки.
- Координаты (широта от –90° до +90°, долгота от –180° до +180°) записываются в ° градусах в виде десятичной дроби.
- Положительные знаки координат – это северная широта и восточная долгота. Отрицательные знаки координат представляются либо знаком «–», южная широта, западная долгота.
- **Выходными** данными будет азимут в ° градусах в виде десятичной дроби и расстояние в километрах.

Комментарии

1. Пусть наш класс выдает допустимые значения:
   от 0 (включая) до 360 (не включая),
   none - для недопустимого азимута (завести перечисление, в котором будет элемент none = -1),
   any - для любого азимута (в имеющееся перечисление, добавить any = -2).
2. Для точек, лежащих на прямой, проходящей через центр Земли азимут = any
3. Для совпадающих точек азимут = none
4. Для одной точки на северном полюсе азимут = 180
5. Для двух точек на одном полюсе азимут = none
6. Для точек на противоположных полюса (см 2) азимут = any
7. Для одной точки на южном полюсе азимут = 180
8. Для точек на экваторе 90 или 270, смотря куда ближе
9. При расчете расстояния между 2 произвольными точками расстояние должно быть меньше Pi*r, где r - радиус Земли. Здесь хорошо бы проверить точки слева и справа от 180-го меридиана
10. Для точек на одном меридиане 0 ("идем на север") или 180 ("идем на юг")